

MedDocs Assistant — Implementation Roadmap (Learning Edition)

A step-by-step build plan for the MedDocs Assistant RAG service (Kotlin + Spring Boot)

- Spring AI + PostgreSQL/pgvector). See [SPEC.md](#) for the *what* and *why*; this document is the *how* and *in what order*.

How to use this doc. The point of this project is to **learn**, not to race to a finished app. Each step has three parts:

- **Build** — what to implement.
- **Learn** — the concepts to genuinely understand before moving on (if you can't explain it out loud, you're not done).
- **Checkpoint** — a concrete, observable proof the step works.

Don't skip a checkpoint. Each stage ends with a **commit/PR boundary** — a clean, reviewable chunk of history, which is itself a hiring signal.

Guiding principles for this build

1. **Make it run before you make it good.** A walking skeleton first; polish later.
 2. **Fake the LLM/embeddings early.** You can build ~80% of the system with a fake embedder that returns deterministic vectors. Swap the real provider in late so you don't burn API credits (or get blocked on a key) while learning the plumbing.
 3. **Write the test with the code, not after.** Especially for the Chunker and the refusal gate — those are the parts interviewers probe.
 4. **Commit per logical step**, not per stage. Small commits = readable history.
 5. **Keep an [OPEN_DECISIONS.md](#)** (or a section in the README) where you record every non-obvious choice as you make it. This becomes your "Design Decisions" section and your interview cheat-sheet.
-

Stage 0 — Tooling & environment (1½ day)

Build

- Install JDK 21 (Temurin), Docker Desktop, and an IDE (IntelliJ IDEA Community is best for Kotlin/Spring).
- Generate the project at [start.spring.io](#):
 - Project: **Gradle - Kotlin**, Language: **Kotlin**, Spring Boot: latest 3.x
 - Dependencies: *Spring Web, Spring Data JPA, Spring Security, Validation, PostgreSQL Driver, Flyway Migration, Docker Compose Support, Lombok?* **no** (Kotlin doesn't need it).
- Commit the generated skeleton + the Gradle wrapper (`./gradlew`).

Learn

- Why a build *wrapper* is committed (`./gradlew` runs the same Gradle version on any machine / CI — no "works on my machine").

- The anatomy of a Spring Boot app: `@SpringBootApplication`, the embedded server, `application.yml` vs `application.properties`.
- Kotlin basics if new: `val/var`, data classes, null-safety (`?`, `!!`, `?:`), no semicolons, trailing-lambda syntax.

Checkpoint

- `./gradlew bootRun` starts the app on port 8080 (it'll fail to find a DB — fine for now).

Commit: `chore: scaffold Kotlin Spring Boot project`

Stage 1 — Skeleton + data model + database (1–2 days)

Build

- `docker-compose.yml` with a **pgvector**-enabled Postgres image (`pgvector/pgvector:pg16`), exposing 5432, with a named volume.
- `application.yml`: datasource pointing at the compose DB; `spring.jpa.hibernate.ddl-auto: validate` (let Flyway own the schema, not Hibernate).
- **Flyway migration V1__init.sql**:
 - `CREATE EXTENSION IF NOT EXISTS vector;`
 - `users, documents, chunks` tables exactly as in `SPEC.md §5`
 - the `embedding vector(384)` column

Model decision (recorded): Answers are generated by **Anthropic Claude** (chat, cloud — uses your one Anthropic account). Embeddings run **locally and free** via Spring AI's in-process `TransformersEmbeddingModel` (`all-MiniLM-L6-v2`, **384 dims**) — no API key, no extra account, no separate service; the document text never leaves your machine (a real medical-privacy talking point). Claude has no embedding model, so the vector dimension is set by the local embedder, not by Claude. Hence `vector(384)`. If you ever swap the *embedder* (not the chat model), this dimension changes and you must re-index.

- JPA entities: `User`, `Document`, `Chunk` (Kotlin classes — careful with JPA + data classes; prefer regular classes with `var` for entities, or use the `kotlin-jpa` plugin to get a no-arg constructor).
- Repositories: `UserRepository`, `DocumentRepository`, `ChunkRepository` (`JpaRepository`).

Learn

- **What pgvector is:** a Postgres extension adding a `vector` type and distance operators (`<=>` cosine, `<->` L2, `<#>` inner product). One DB for relational + vector — your headline design decision.
- **Migrations vs Hibernate auto-DDL:** why production schemas are versioned files (Flyway), and why `ddl-auto: validate` (not `update`) is the safe choice.
- **JPA/Hibernate basics:** entity ↔ table mapping, `@Entity`, `@Id`, `@GeneratedValue`, `@ManyToOne/@OneToMany`, the N+1 problem (just be aware).
- The Kotlin-JPA friction (final classes, no-arg ctor) and how the `kotlin-jpa` / `kotlin-allopen` plugins fix it.
- **Two engines, two roles:** Claude (cloud chat) generates answers; a local ONNX model (in-process embeddings) turns text into vectors. The `vector(n)` dimension is decided by the *embedder* (MiniLM = 384), never by the chat model.

Checkpoint

- `docker compose up -d` → DB is up.
- `./gradlew bootRun` → Flyway applies **V1**, Hibernate **validate** passes, app starts clean. `\d chunks` in psql shows the **vector(384)** column.

Commit: `feat: data model, pgvector schema, flyway migration`

Stage 2 — Auth: register / login / JWT (2 days)

Build

- `POST /api/auth/register` — validate email + password, BCrypt-hash, save **User**.
- `POST /api/auth/login` — verify credentials, return a signed **JWT**.
- Spring Security config: stateless (`SessionCreationPolicy.STATELESS`), a `JwtAuthenticationFilter` that reads **Authorization: Bearer <token>**, validates it, and sets the **SecurityContext**.
- All `/api/**` require auth **except** `/api/auth/**`.
- A protected `GET /api/ping` returning the current user's email — your test target.
- JWT secret from an **env var** (`.env.example` documents it).

Learn

- **What a JWT actually is:** header.payload.signature, base64url, signed (not encrypted!) — so never put secrets in the payload. HMAC vs RSA signing.
- **Stateless auth:** why there's no server session; the token is the session.
- The Spring Security **filter chain** mental model: request → filters → **SecurityContext** → controller. Where your JWT filter slots in.
- **BCrypt:** salting, work factor, why you never store or compare plaintext.
- Trade-off you should be able to defend: short expiry + no refresh token (simpler, fine for a demo) vs refresh tokens (better UX, more moving parts). **Pick & note it.**

Checkpoint

- `GET /api/ping` → **401** without a token, **200** + your email with a valid token.
- Write the **web-layer test** now: `@WebMvcTest` or `@SpringBootTest` proving 401/200.

Commit: `feat: jwt auth, spring security, protected ping + test`

Stage 3 — Ingestion: upload → parse → chunk → embed → store (3–4 days)

This is where the RAG-specific learning starts. Build it with a **fake embedder** first.

Build (3a) — Upload & parse

- `POST /api/documents` (multipart) — accept PDF/TXT/MD, validate size + mime type, save a **Document** row with **status** = **PENDING** and the **source_label** field (default to filename if blank).
- **DocumentParser:** extract plain text. Use Apache **Tika** (Spring AI's PDF reader wraps it) for PDF; trivial for TXT/MD.

Build (3b) — Chunker (write this yourself + unit-test it)

- **Chunker**: split text into overlapping windows (~800 "tokens", ~100 overlap).
- Decide & document your unit: real tokens via Spring AI **TokenTextSplitter** (uses jtokkit), or characters/words approximating tokens. **Name the choice.**
- Pure function, **zero Spring**, fully unit-tested.

Build (3c) — Embedding & storage

- **EmbeddingService** behind an interface **Embedder**. Ship a **FakeEmbedder** first (deterministic vector from text hash) so the whole pipeline runs with no API key.
- For each chunk: embed → store **content** + **section_ref** + **embedding** + **chunk_index**.
- Mark **Document** **INDEXED**, or **FAILED** with a reason. Make it **all-or-nothing** (one transaction): if any chunk fails, the doc is **FAILED**, no partial index.
- Run ingestion **async** (**@Async** method) so the upload returns immediately and the UI polls **GET /api/documents** for status.

Learn

- **Why chunk at all**: embedding models + LLM context windows have limits; retrieval granularity matters. The **size/overlap trade-off**: too big = noisy retrieval & wasted context; too small = lost meaning; overlap = preserves cross-boundary context.
- **What an embedding is**: text → a fixed-length vector where semantic similarity ≈ geometric closeness. Why the **same model must embed both docs and queries**.
- The **embedding-dimension coupling** (your **vector(384)** = **all-MiniLM-L6-v2**): swapping the **embedder** (e.g. to Ollama **nomic-embed-text** 768-dim or OpenAI 1536-dim) breaks the schema and invalidates every stored vector → needs a migration + full re-index. Note this is about the **embedding** model only — changing the **chat** model (Claude → another LLM) does *not* touch the vector column. Understand this deeply; it's a guaranteed interview question.
- **Async + status lifecycle**: why long work doesn't belong in the request thread.
- The **interface-behind-the-AI** pattern (**Embedder**) — this is what makes the core unit-testable and the provider pluggable.

Checkpoint

- Upload a TXT → **GET /api/documents** shows **PENDING** then **INDEXED**.
- **SELECT count(*) FROM chunks WHERE document_id = ... > 0**; embeddings populated.
- **ChunkerTest** green: boundaries, overlap correctness, empty input, huge input.

Commit: feat: ingestion pipeline (parse, chunk, embed, store) + chunker tests

Stage 4 — RAG core: retrieve → threshold/refuse → prompt → answer (4–5 days)

The heart of the project. Go slow. Write it yourself. This is what you'll whiteboard.

Build (4a) — Vector search

- **vector/** repository: native query **SELECT ... 1 - (embedding <=> :queryVec) AS score ... ORDER BY embedding <=> :queryVec LIMIT :k**, **scoped to the user's documents** (per-user isolation —

non-negotiable).

- Returns chunks **with similarity scores**.

Build (4b) — The refusal gate (the headline feature)

- After retrieval, check the **best score** against a configured threshold.
- If below threshold → return `{ grounded: false, answer: "<refusal message>", sources: [] }` and **never call the LLM**.
- Be precise about the metric: pgvector `<=>` is cosine **distance** (0=identical, 2=opposite); similarity = `1 - distance`. State whether the threshold is on similarity or distance, and pick a starting value (e.g. similarity ≥ 0.75) you can tune. Make it a config property.

Build (4c) — Grounded prompt assembly

- `RagService` (interface-driven, framework-light) builds the prompt:
 - **system instruction**: "Answer ONLY from the provided context. Cite sources. If the context is insufficient, say you don't know." (the anti-hallucination instruction)
 - the retrieved passages (with their `source_label/section_ref`)
 - the user's question
- Call the LLM via Spring AI `ChatClient` (still mockable). Return `{ grounded: true, answer, sources[] }` where `sources = {documentId, sourceLabel, sectionRef, snippet, score}`.

Build (4d) — `POST /api/query`

- Wire `{question}` → embed → search → gate → (refuse | prompt + LLM) → response.

Learn

- **The full RAG loop** end to end — be able to draw it from memory.
- **Cosine distance vs similarity** and why `ORDER BY embedding <=> q` gives nearest neighbors.
- **Prompt grounding / context injection**: how the retrieved text constrains the model, and *why it reduces but doesn't eliminate* hallucination (the model can still err *within* context — so "guarantee" is too strong; it's a *mechanism*).
- **Why the refusal path matters in a medical context**: a grounded "I don't know" beats a confident wrong answer. This is the project's whole thesis.
- **Testing AI code without AI**: mock `ChatClient` + `Embedder` so tests are fast, deterministic, and free.

Checkpoint — the tests that define this project

- `RagServiceTest` (mocked chat + embedding clients) proves:
 1. retrieved context **is injected** into the prompt,
 2. the "answer only from context / cite sources" instruction **is present**,
 3. **below-threshold retrieval triggers refusal and the LLM is NEVER called** (verify with `verify(chatClient, never())...`). ← the headline test.
- Manual: ask an in-corpus question → cited answer; ask an out-of-scope question → grounded refusal.

Commit: `feat: rag core – vector search, refusal gate, grounded prompt + tests`

Stage 5 — Swap in the real AI provider + integration test (1–2 days)

Build

- Implement the real **chat** call via Spring AI's **Anthropic** starter (Claude), keyed by `ANTHROPIC_API_KEY` (your one account). Implement the real **Embedder** via Spring AI's in-process `TransformersEmbeddingModel` (`all-MiniLM-L6-v2`, ONNX) — **no API key, runs locally**. Keep `FakeEmbedder` for tests.
- The MiniLM model file downloads once on first run; after that it's fully offline.
- **Re-index** any docs embedded with the fake embedder (dimensions/space differ).
- **Integration test** with **Testcontainers** (real Postgres+pgvector in a container)
 - a stubbed LLM: prove retrieval returns the right chunks and **user isolation holds** (user A can never retrieve user B's chunks).

Learn

- **Spring AI's abstractions:** `ChatClient` (→ Claude, **remote**) and `EmbeddingModel` (→ local ONNX MiniLM, **in-process**) are *independent* — wired side by side, auto-configured from properties. One is a typed HTTP client to the cloud; the other runs the model right inside your JVM. Same interface, very different execution.
- **Testcontainers:** spinning real infra in tests for fidelity vs mocks for speed — when to use which.
- **Cost awareness:** add a crude per-user query cap or you risk an open demo draining API credits (responsible-engineering signal).

Checkpoint

- A real question over a real embedded doc returns a real, cited answer.
- Integration test green; isolation test proves cross-user leakage is impossible.

Commit: `feat: real OpenAI provider via spring ai + testcontainers integration test`

Stage 6 — Frontend (Vue 3 + TS) (3–4 days)

Build

- Vite + Vue 3 + TS app: login/register, document upload (with `source_label`), document list with live **status** (poll while `PENDING`), and a chat view.
- Chat view renders the answer, the `sources[]` (label + section + snippet + score), **and the refusal state clearly** when `grounded: false`.
- Store the JWT (memory or `localStorage` — note the XSS trade-off), attach it as a `Bearer` header on every API call.
- CORS config on the backend for the dev origin.

Learn

- **SPA ↔ JWT auth flow:** where the token lives, how it's attached, why CORS exists.
- Vue 3 Composition API basics (`ref`, `computed`, `onMounted`), `fetch`/axios, rendering async state (loading / error / refusal / answer).
- Designing UI that **makes refusal a first-class, non-scary state** — part of the responsible-AI story.

Checkpoint

- Full happy path in the browser: register → upload → see INDEXED → ask → cited answer; and an out-of-scope question → visible grounded refusal.

Commit(s): `feat(ui): auth, upload, document list, chat with sources + refusal state`

Stage 7 — CI, deploy, README, demo (2–3 days)

Build

- **GitHub Actions:** on push/PR → `./gradlew clean build`, with Postgres+pgvector as a service container for the integration tests. Add the status **badge** to the README.
- **Deploy** to Railway or Render with managed Postgres (verify pgvector support *before* committing to a provider). Secrets (LLM key, DB URL, JWT secret) via env vars.
- **Preload a small public corpus** so the live demo answers immediately. (Resolve the corpus-visibility question first — see Open Decisions below.)
- **README** in the order from `SPEC.md §11`: problem + live link + disclaimer → demo GIF (answer **and** refusal) → architecture diagram → tech stack with one-line whys → **Design Decisions** → run locally → run tests → limitations / next.
- Record the **demo GIF** showing both a cited answer and a grounded refusal.

Learn

- **CI as a contract:** every push proves the build + tests pass on a clean machine.
- **12-factor config / secrets via env**, never in the repo.
- **Writing a README that sells judgment**, not just lists commands — the Design Decisions section is what gets you the interview.

Checkpoint — Definition of Done (`SPEC.md §13`)

- ☐ Live URL: a stranger registers, asks, gets a cited answer + sees a grounded refusal.
- ☐ `./gradlew clean build` green; Actions badge passing.
- ☐ README complete: diagram, GIF, Design Decisions, disclaimer.
- ☐ `docker-compose up` brings it all up from a clean clone.
- ☐ No secrets in the repo; `.env.example` present.
- ☐ You can whiteboard the full flow — including the refusal gate — from memory.

Commit: `ci: github actions + deploy + README + design decisions`

Open decisions to resolve as you go (record answers in README/OPEN_DECISIONS.md)

These are the "prove you have judgment" points. Don't leave them implicit.

1. **Corpus visibility** — how does a brand-new user query the *preloaded public corpus* while per-user isolation still holds? (Recommended: a shared/global corpus via a `visibility/nullable-user_id` column, plus private user uploads.)
2. **Embedding dimension coupling** — `vector(384)` is fixed to `all-MiniLM-L6-v2`; switching *embedders* requires a migration + full re-embed. Switching the *chat* model (Claude → other) does not. State both.

Claude has no embedding model — that's why answers (cloud Claude) and embeddings (local ONNX) come from two different engines. Document the deliberate choice to embed locally (free, private, offline).

3. **What "citation" means** — are `sources[]` the *retrieved* top-k passages (simple, honest) or *true inline* `[1][2]` markers the model emits and you parse (fancier, more failure modes)? Don't claim "inline" if you ship the list.
4. **Refusal threshold metric & value** — similarity vs distance, and the starting number, plus how you'd tune it.
5. **Chunking unit** — real tokens (which tokenizer) or char/word approximation.
6. **JWT lifetime** — short-expiry single token vs refresh tokens.
7. **Cost guard** — per-user query/rate cap for the public demo.

Suggested learning resources

- **Spring AI** reference docs (ChatClient, EmbeddingModel, the pgvector VectorStore).
- **pgvector** README (distance operators, indexing: exact vs HNSW/IVFFlat).
- **Spring Security** "Architecture" + "JWT" sections (the filter-chain model).
- **Kotlin** official docs "Kotlin for Java developers" + the `kotlin-spring/kotlin-jpa` plugin notes.
- **Testcontainers** Spring Boot guide.
- A short read on **RAG fundamentals** (retrieval → grounding → generation) to anchor the vocabulary before Stage 4.

Rough time budget (learning pace, part-time)

Stage	Focus	Est.
0	Tooling	½ day
1	Data model + pgvector	1–2 d
2	Auth + JWT	2 d
3	Ingestion + Chunker	3–4 d
4	RAG core + refusal gate	4–5 d
5	Real provider + integration tests	1–2 d
6	Frontend	3–4 d
7	CI + deploy + README	2–3 d

Total: ~3–4 focused weeks part-time. The two stages worth slowing down on are **Stage 4 (RAG core)** and **Stage 2 (auth)** — they're the most-probed in interviews.